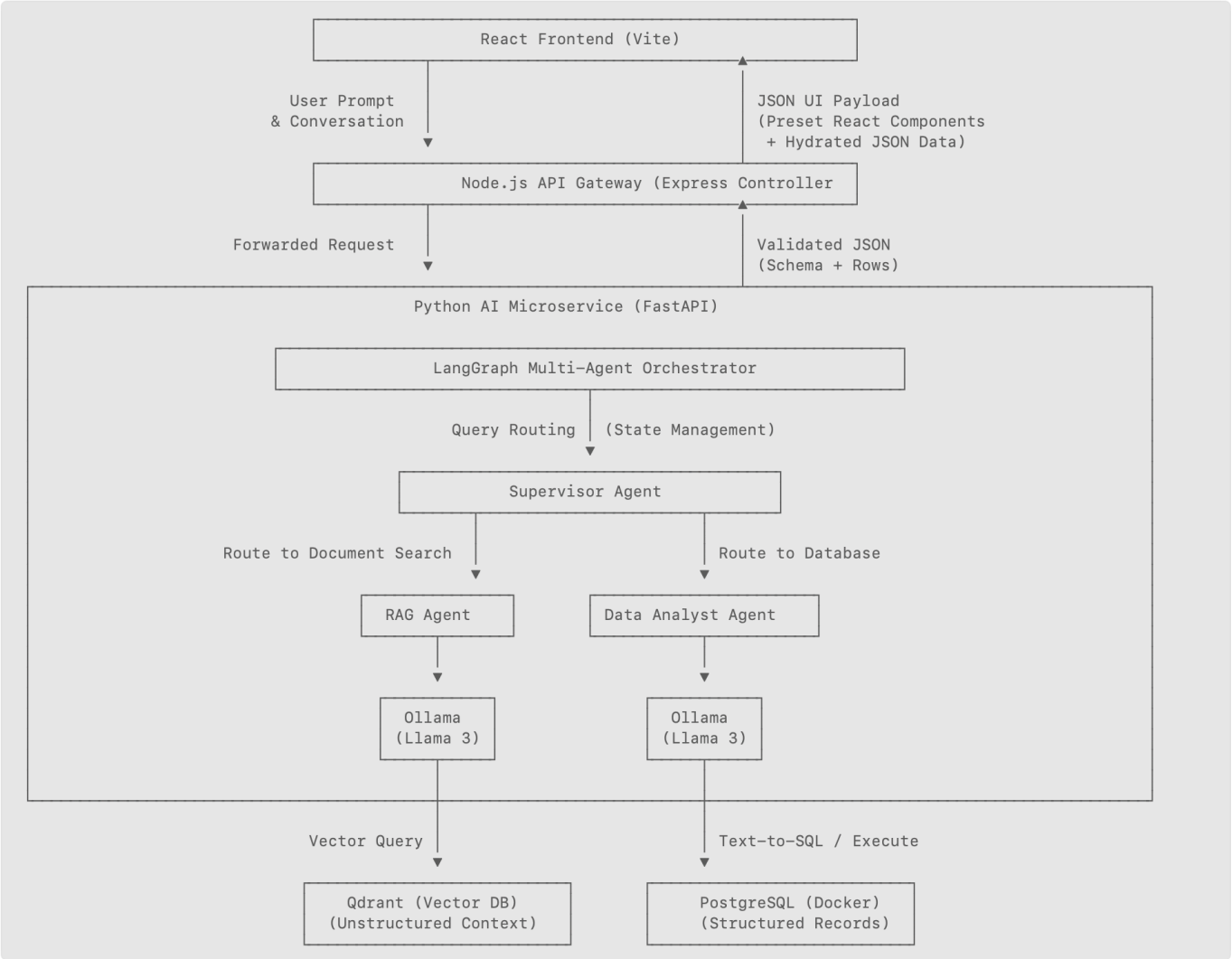
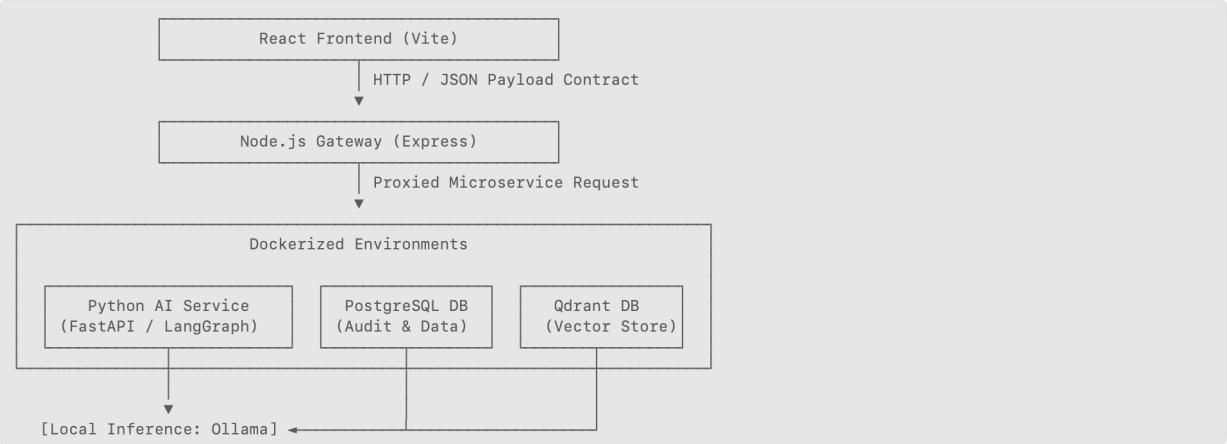
Enterprise Autonomous Analytics & Knowledge Engine

This is an enterprise-grade, full-stack AI platform built on a decoupled microservices architecture. The system features an intelligent LangGraph Multi-Agent Orchestrator, a Production RAG Pipeline backed by Qdrant, automated Text-to-SQL data synthesis via PostgreSQL, and a deterministic Declarative UI Engine built with React and Node.js.

The system operates entirely on local, open-source LLMs (Ollama/Llama 3), ensuring zero API costs, total data privacy, and predictable deterministic UI layouts using strict JSON schema validation contracts.

Enterprise Architecture Diagram





Technology Stack

Frontend

- React 18
- Vite
- TypeScript
- Tailwind CSS

Backend API Gateway

- Node.js
- Express
- TypeScript
- pg (PostgreSQL client/pool)

AI Service

- Python
- FastAPI
- Uvicorn
- LangGraph
- Qdrant Python Client
- Sentence Transformers (all-MiniLM-L6-v2)
- psycopg2-binary

Data and Infra

- PostgreSQL
- Qdrant
- Docker Compose

LLM Runtime

- Ollama
- Llama 3

Swagger

- Swagger UI: http://localhost:8000/docs
- The FastAPI service (port 8000) has built-in Swagger UI automatically enabled.

- Swagger API: http://localhost:5001/api-docs
- OpenAPI 3.0 specification
- Endpoints:
  * POST /api/gateway/chat — AI query submission
  * GET /api/gateway/audit — System audit metrics
* Detailed request/response schemas and examples

The endpoints can be tested directly from the Swagger UI interface.

Key Design Patterns Implemented

- **Declarative Generative AI UI**: Eliminates unpredictable raw Markdown LLM outputs. The Python service returns a structured, type-safe JSON payload that instructs the React client precisely which pre-registered visual component to render (REGULAR_DATA_TABLE or METRIC_CARD_GRID).

- **LLM-Powered Supervisor Routing**: Bypasses simplistic keyword routing. Uses a local LLM instance to dynamically evaluate semantic prompt intent and branch state workflows.

- **Resilient Database Connection Pooling**: Minimizes resource overhead via robust connection pools (SimpleConnectionPool in Python and Pool in Node.js) with explicit transaction rollback context managers.

- **Telemetry and Tracing Observability**: Implements a middleware auditing filter logging prompt ingestion metadata, targeted interface variants, and execution latencies directly into database tables.

Tech Stack & Core Dependencies

Frontend (/frontend)

- **React 18 & Vite**: Component-driven architecture utilizing hot module reloading.
- **TypeScript**: Strict type boundaries matching the global schema contract.
- **Tailwind CSS:** Fast utility styling layouts for application elements.

Backend API Gateway (/backend)

- **Node.js (Express + TypeScript)**: Secure infrastructure shield handling authentication and data auditing.
- **PG (pg)**: High-performance PostgreSQL pooling abstraction layer client.

Python AI Microservice (/ai-service)

- **FastAPI & Uvicorn**: High-throughput, asynchronous ASGI execution framework.
- **LangGraph**: State-driven multi-agent execution graphs with conditional branching.
- **Qdrant Python Client & Sentence Transformers**: Free local text embedding orchestration (all-MiniLM-L6-v2) and geometric vector persistence.
- **Psycopg2-Binary**: Production-ready data driver interface mapping database actions.

Engineering Challenge: Solving LLM Unpredictability with Declarative UI

- **The Situation**: In many AI applications, developers rely on the LLM to output raw Markdown text or unstructured HTML tables. During user testing, I realized this creates massive layout fragmentation because LLMs are non-deterministic and frequently hallucinate missing tags, which can completely crash the client UI.
- **The Task**: I needed to enforce absolute stability on the frontend while still allowing the generative AI engine the freedom to query different datasets dynamically.
- **The Action**: Instead of letting the model generate raw presentation code, I implemented a **Declarative Generative AI pattern**. I created a strict JSON schema contract and shared it across the entire stack. I built custom Pydantic validation models in FastAPI and built a dedicated UniversalAIRenderer registry in React. The AI model is strictly forced to output raw data and structural metadata. React reads that schema and dynamically selects a pre-coded, type-safe frontend component to handle the layout securely.
- **The Result**: This decoupled architecture guaranteed a 100% stable user experience. The model can change table headers or data types on the fly, but it can never break the browser's presentation layer.

Engineering Challenge: Intelligent Routing and Graph State Management

- **The Situation**: A major problem with early-stage AI systems is routing queries efficiently. Hardcoded if/else keyword checks fail when user phrasing changes, but sending every single request through a massive model chain increases latency and burns compute tokens.

- **The Task**: I needed an intelligent orchestrator capable of evaluating semantic intent and conditionally branching to separate, specialized database micro-agents.

- **The Action**: I used **LangGraph** to construct a state-driven multi-agent workflow. I decoupled the routing logic into a dedicated **LLM-powered Supervisor Agent** running locally on an Ollama model instance. The supervisor acts as a fast traffic cop: it samples user intent and routes unstructured document queries to a localized **Qdrant Vector DB (RAG)** node, while routing quantitative metric queries to a **PostgreSQL Text-to-SQL data analyst** agent.

- **The Result**: This optimization restricted expensive model operations only to nodes that actually required them. I also implemented defensive programming with an asynchronous try/except string-matching backup so that if the local LLM runtime ever went offline, the graph cleanly executed a static fallback route instead of throwing an unhandled exception.

Engineering Challenge: Production Database Practices & Connection Pooling

- **The Situation**: Many developers use default database wrappers like SQLite or create a fresh, unmanaged connection every time an API endpoint is hit. In an enterprise system under heavy user load, spinning up hundreds of concurrent TCP database connections quickly exhausts server file descriptors and crashes the database.
- **The Task**: I needed to ensure that our structured PostgreSQL database and unstructured Qdrant vector database could scale securely without resource leaks.

- **The Action**: I isolated both data stores into dedicated container networks using **Docker Compose** with custom health check scripts. In the Node.js API gateway, I utilized the pg library to implement a persistent **Connection Pool** configured with explicit timeout boundaries. In the Python service layer, I wrote custom context managers (with statements) using SimpleConnectionPool to ensure every single data cursor leased to an AI agent was automatically released back to the cluster pool, even during execution crashes.

- **The Result**: This isolated container architecture completely eliminated the risk of hanging connections, prevented memory leaks, and allowed the backend infrastructure to safely withstand heavy, concurrent AI workloads.

Engineering Challenge: System Observability, Monitoring, and Telemetry

- **The Situation**: AI agent networks can be opaque black boxes. When an application experiences slowdowns, it is incredibly difficult to diagnose whether the bottleneck is caused by a slow database query, network transit lag, or local model generation latency.
- **The Task**: I needed a way to log, trace, and monitor system health to guarantee operational visibility.
- **The Action**: I built a customized **Telemetry Filter** inside the Node.js Express middleware gateway. Every time an interaction hits the API, the system spins up a high-precision performance execution timer. Before the request is forwarded, the client prompt is captured. Once the microservice response completes, the gateway logs the exact user prompt, the target declarative UI layout returned, and the total transit latency directly into a persistent audit_logs table in PostgreSQL. I then built an administrative dashboard view in React that polls this telemetry endpoint to show real-time KPIs and average latency tracking maps.
- **The Result**: This gave me immediate visibility into the agent loops. If an AI model starts generating malformed queries or taking too long to reply, I can pinpoint the latency spike instantly on my dashboard.

By running the embedding calculations (all-MiniLM-L6-v2) and model generation loops (Llama 3/Mistral) locally on an **Ollama** engine, the enterprise saves thousands of dollars in commercial API fees. This makes the architecture completely private and highly cost-effective to prototype and scale.

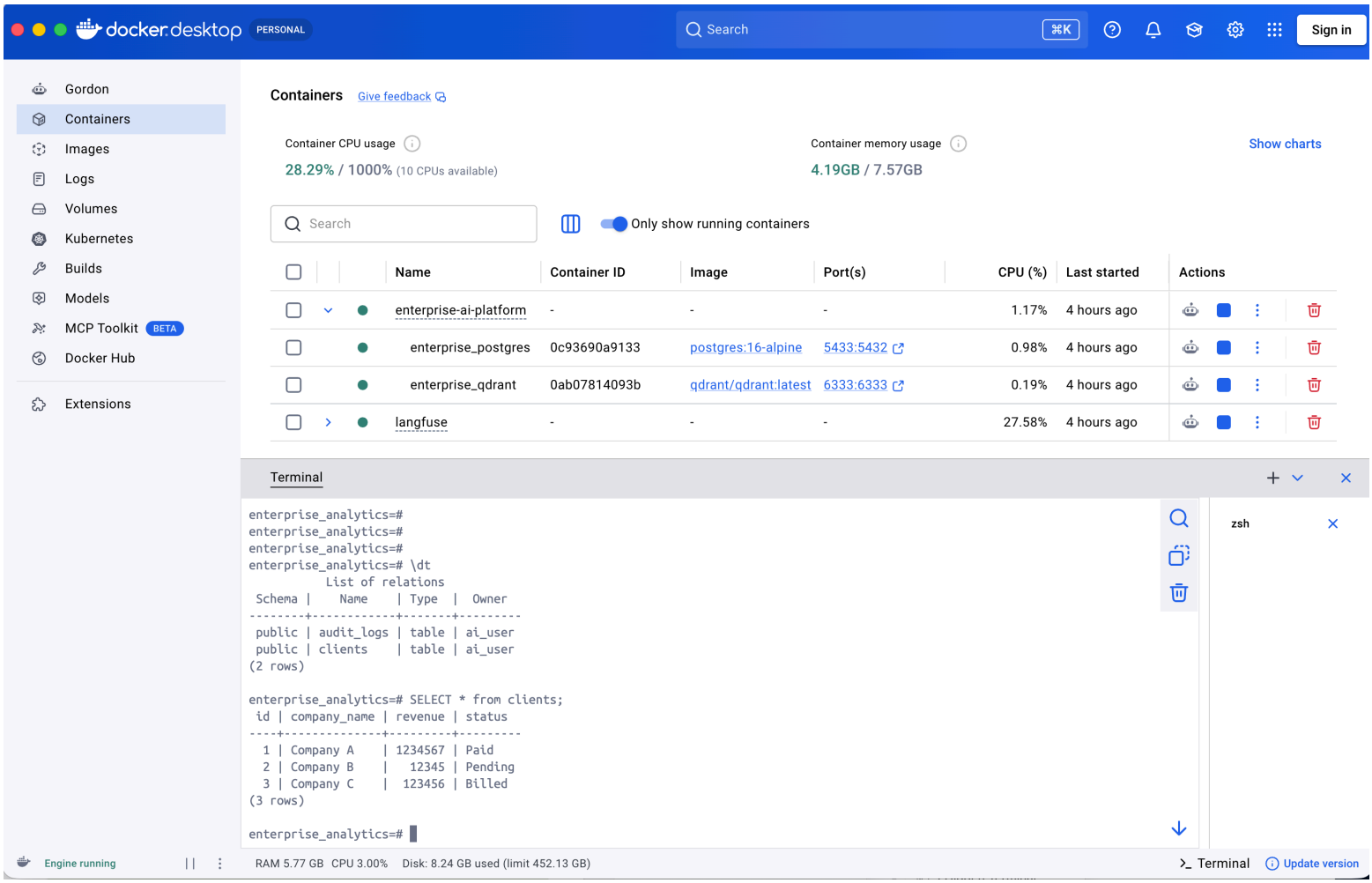
ENTERPRISE-AI-PLATFORM Tree Structure

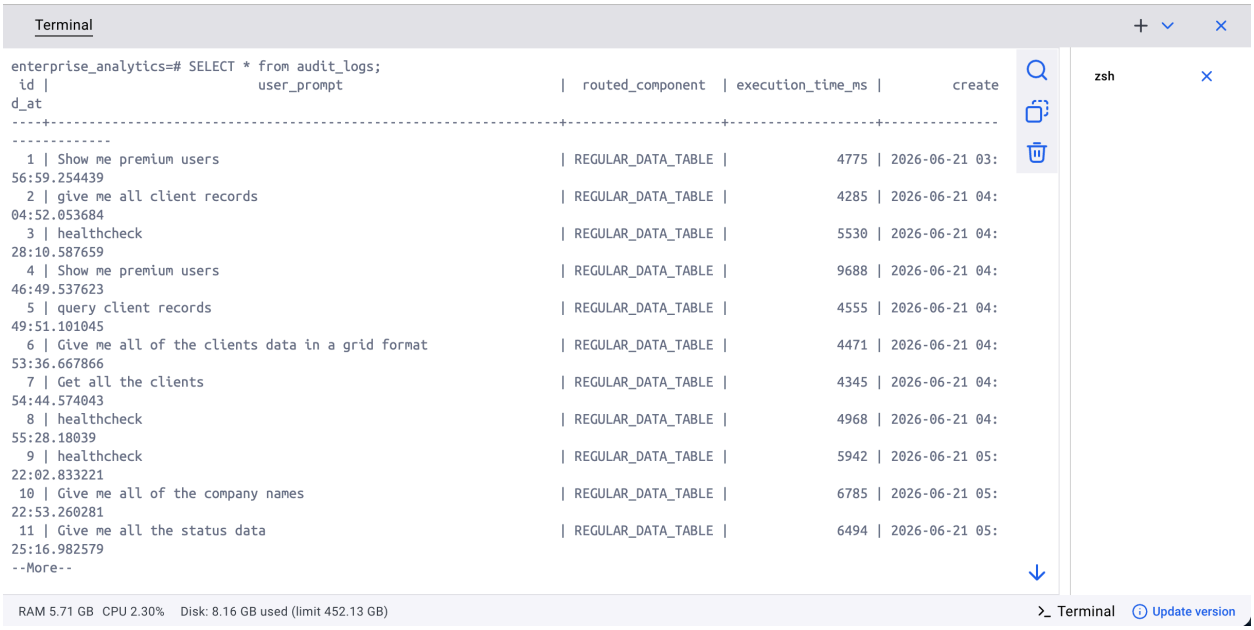
(excluding generated folders like node_modules, venv, dist, **pycache**, and .DS_Store)

```
.vscode
|-- settings.json
ai-service
|-- main.py
|-- requirements.txt
|-- app
| |-- agents
| | |-- analyst_agent.py
| | |-- graph.py
| | |-- rag_agent.py
| | |-- supervisor.py
| |-- database
| | |-- init.sql
| | |-- pg_client.py
| | |-- qdrant_client.py
| |-- schemas
| |-- output_models.py
```

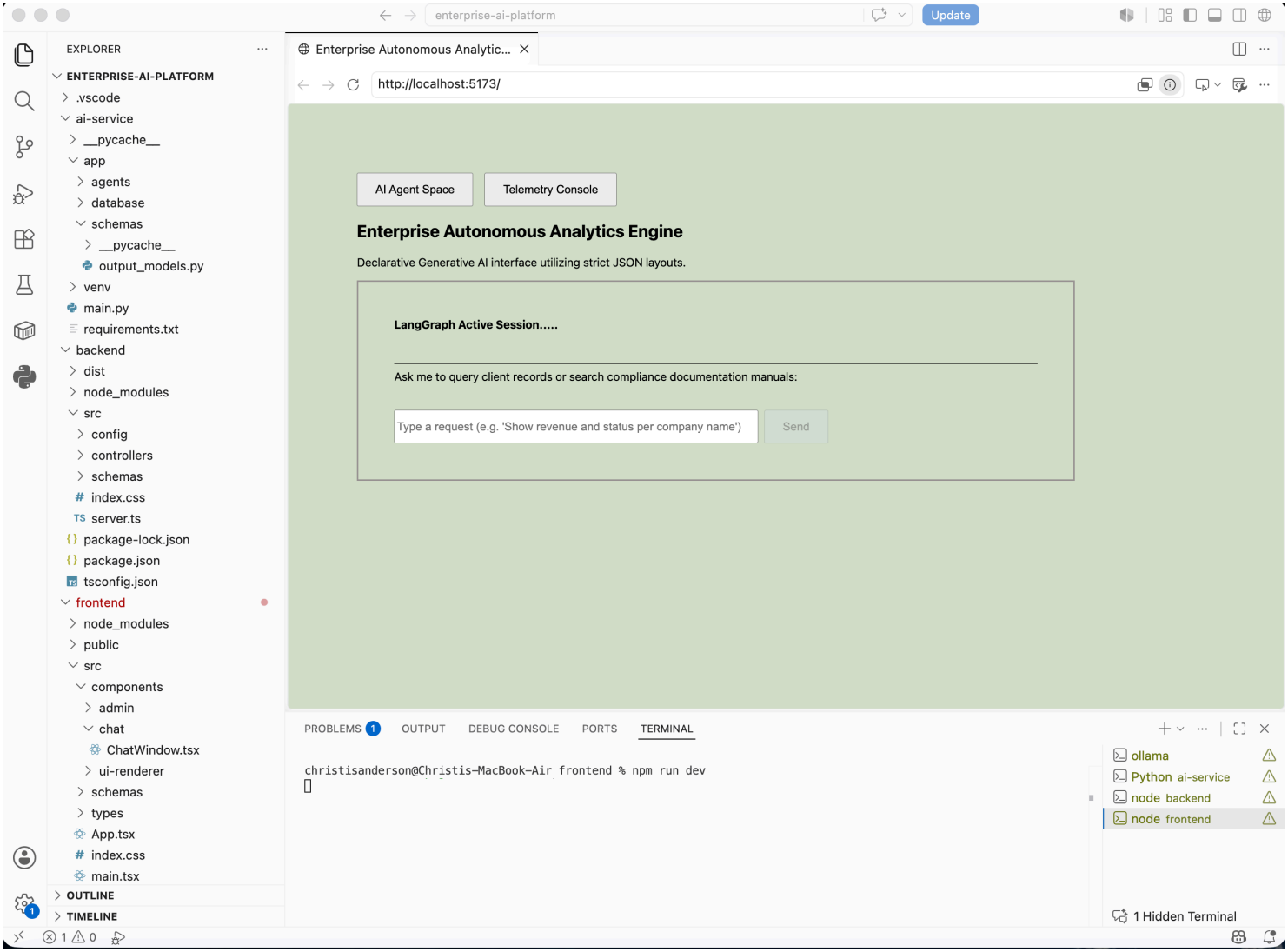
```
backend
|-- package.json
|-- tsconfig.json
|-- src
| |-- index.css
| |-- server.ts
| |-- config
| | |-- db.ts
| |-- controllers
| | |-- auditController.ts
| | |-- queryController.ts
| |-- schemas
| |-- contract-schema.json
frontend
|-- index.html
|-- package.json
|-- tailwind.config.js
|-- tsconfig.json
|-- tsconfig.node.json
|-- public
|-- src
| |-- App.tsx
| |-- index.css
| |-- main.tsx
| |-- vite-env.d.ts
| |-- components
| | |-- admin
| | | |-- adminDashboard.tsx
| | |-- chat
| | | |-- ChatWindow.tsx
| | |-- ui-renderer
| | |-- DynamicRenderer.tsx
| | |-- tables
| | |-- MetricCardGrid.tsx
| | |-- RegularDataTable.tsx
| |-- schemas
| | |-- contract-schema.json
| |-- types
| |-- ai-response.ts
| |-- audit.ts
docker-compose.yml
README.md
```

Dump PostgreSQL tables in DOCKER:

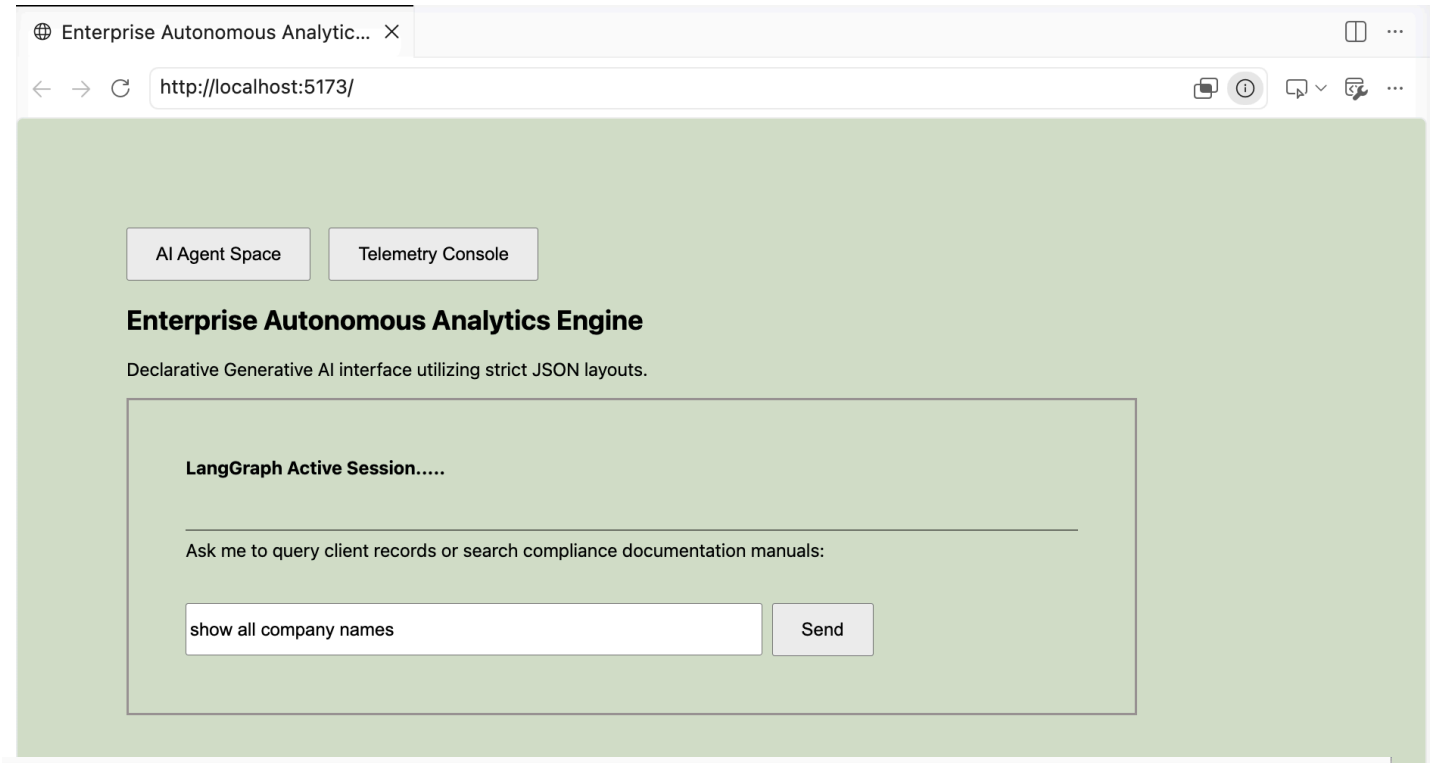


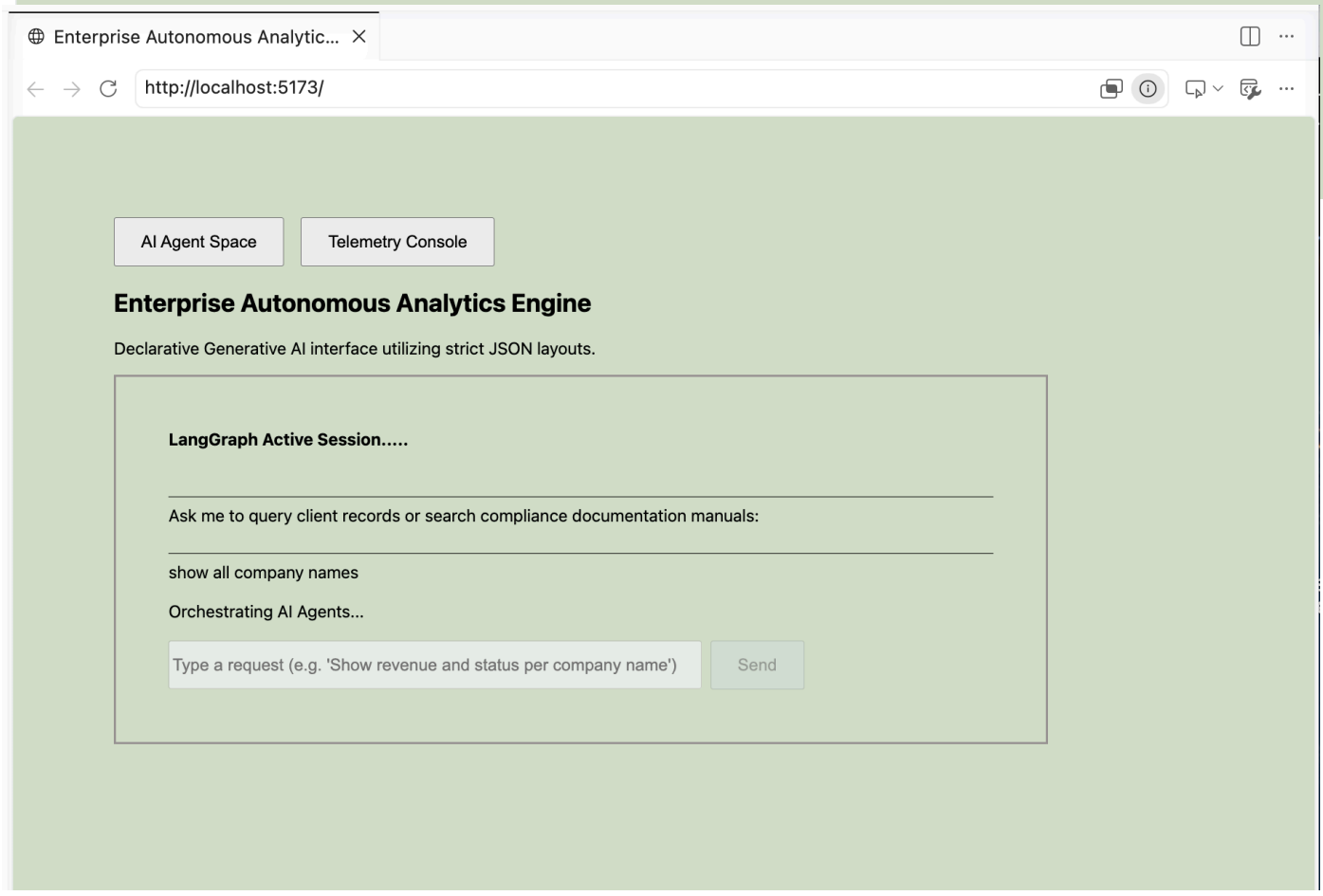


DEMO: AI Agent Space Initial State



Send requests to Supervisor Agent so it passes to the Data Analyst Agent (PostgreSQL in Docker) and sends back a Declarative AI structured response from the clients table:





AI Agent Space | Telemetry Console

Enterprise Autonomous Analytics Engine

Declarative Generative AI interface utilizing strict JSON layouts.

LangGraph Active Session.....

Ask me to query client records or search compliance documentation manuals:

---

show all company names

Pipeline executed successfully. Rendered structural view variant: REGULAR_DATA_TABLE

Live DB Stream

Company Name
Company A
Company B
Company C

Type a request (e.g. 'Show revenue and status per company name')

Send

AI Agent Space | Telemetry Console

Enterprise Autonomous Analytics Engine

Declarative Generative AI interface utilizing strict JSON layouts.

LangGraph Active Session.....

Ask me to query client records or search compliance documentation manuals:

---

show all company names

Pipeline executed successfully. Rendered structural view variant: REGULAR_DATA_TABLE

Live DB Stream

Company Name
Company A
Company B
Company C

Show revenue and status by company name

Orchestrating AI Agents...

Type a request (e.g. 'Show revenue and status per company name') | Send

AI Agent Space | Telemetry Console

Enterprise Autonomous Analytics Engine

Declarative Generative AI interface utilizing strict JSON layouts.

LangGraph Active Session.....

Ask me to query client records or search compliance documentation manuals:

---

show all company names

Pipeline executed successfully. Rendered structural view variant: REGULAR_DATA_TABLE

Live DB Stream

Company Name
Company A
Company B
Company C

Show revenue and status by company name

Pipeline executed successfully. Rendered structural view variant: REGULAR_DATA_TABLE

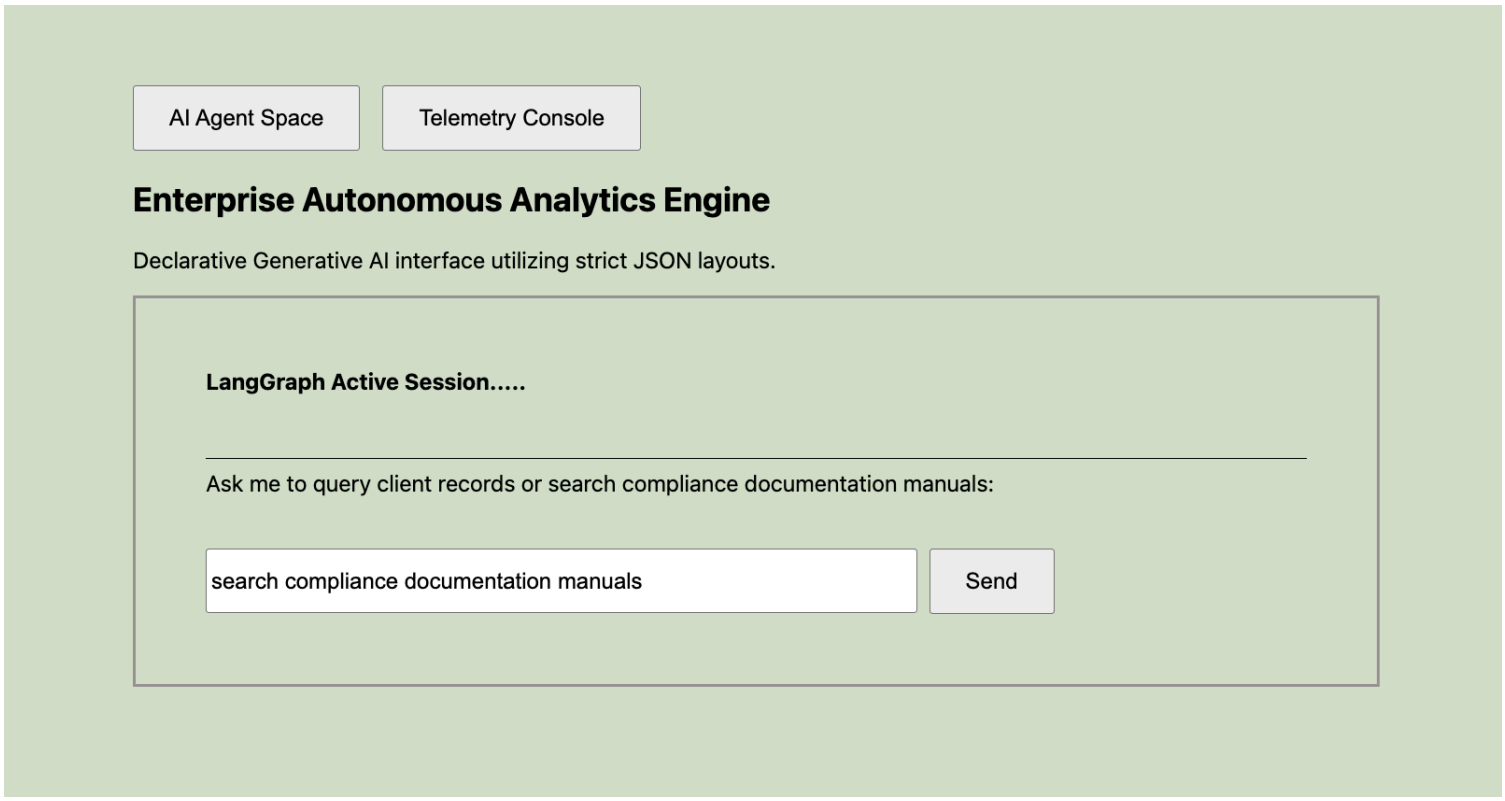
Analytics Response (Procedural Fallback Mapping)

Live DB Stream

Company Name	Revenue	Status
Company A	$1,234,567.00	Paid
Company B	$12,345.00	Pending
Company C	$123,456.00	Billed

Type a request (e.g. 'Show revenue and status per company name') | Send

Send requests to Supervisor Agent so it passes to the RAG Agent (Qdrant Vector DB) to render the unstructured content:



AI Agent Space | Telemetry Console

Enterprise Autonomous Analytics Engine

Declarative Generative AI interface utilizing strict JSON layouts.

LangGraph Active Session.....

Ask me to query client records or search compliance documentation manuals:

---

search compliance documentation manuals

Pipeline executed successfully. Rendered structural view variant: REGULAR_DATA_TABLE

Knowledge Base Context Search: 'search compliance documentation manuals'

Live DB Stream

Match Confidence Score	Source File	Matching Excerpt
0.1939	compliance_2026.txt	Data Management Mandate: Financial logging metrics must persist for 7 years.
0.146	infra_specs.pdf	System Architecture Guidelines: Use database indexing on client identity rows.

Type a request (e.g. 'Show revenue and status per company name') | Send

Click the Telemetry Console button and send request to Supervisor Agent so it passes to the Data Analyst Agent (PostgreSQL in Docker) and sends back a Declarative AI structured response from the audit_logs table: